LLM. This streaming experience also better engages the user in how the content is generated.

Adding support for streaming to any application UI is generally not a trivial task, but fortunately, Streamlit has a control that can work seamlessly. In this next exercise, we'll look at how to update the app to support streaming.

Open `chapter_7/chatgpt_clone_streaming.py` in VS Code. The relevant updates to the code are shown in listing 7.6. Using the `st.write_stream` control allows the UI to stream content. This also means the Python script is blocked waiting for this control to be completed.

Listing 7.6 `chatgpt_clone_streaming.py` (relevant section)

```
with st.chat_message("assistant"):
    stream = client.chat.completions.create(
        model=st.session_state["openai_model"],
        messages=[
            {"role": m["role"], "content": m["content"]}
            for m in st.session_state.messages
        ],
        stream=True,     #1
    )
    response = st.write_stream(stream)     #2
st.session_state.messages.append(
{"role": "assistant", "content": response})      #3
```

#1 Sets stream to True to initiate streaming on the API
#2 Uses the stream control to write the stream to the interface
#3 Adds the response to the message state history after the stream completes

Debug the page by pressing F5 and waiting for the page to load. Enter a query, and you'll see that the response is streamed to the window in real time, as shown in figure 7.6. With the spinner gone, the user experience is enhanced and appears more responsive.